🎨 CSS @property Rule

Define and control typed CSS custom properties with advanced animations

What is the CSS @property Rule?

The `@property` rule is an advanced CSS feature that allows you to register custom properties (CSS Variables) with specific types, initial values, and inheritance rules. This enables smoother animations and transitions of custom properties that would otherwise not be animatable.

Unlike regular CSS Variables that are treated as strings, typed properties with `@property` can be animated smoothly because the browser understands their data type and can interpolate between values.

Browser Support: Chrome 85+, Edge 85+, Safari 16.4+, Firefox (limited). Support is still emerging, so use feature detection and fallbacks.

Core Concepts & Syntax

@property Rule Syntax

The @property rule registers a typed custom property with specific metadata:

```
@property --property-name {
  syntax: '<type>';
  inherits: true | false;
  initial-value: <value>;
}
```

The syntax Descriptor

Defines the data type of the custom property:

```
Common Syntax Values:

/* Colors */
syntax: '<color>';

/* Numbers and Angles */
syntax: '<number>';
syntax: '<integer>';
syntax: '<angle>';
syntax: '<length>';

/* Percentages */
syntax: '<percentage>';

/* Anything (fallback) */
syntax: '*';

/* Multiple types */
syntax: '<length> | <percentage>';
syntax: '<color>#';
syntax: '<number>+';
```

The inherits Descriptor

Specifies whether the property is inherited to child elements:

```
inherits: true
/* Property inherits from parent like regular CSS properties */

inherits: false
/* Property doesn't inherit - each element has independent value */
```

The initial-value Descriptor

Sets the default value if the property is not explicitly set:

```
@property --my-color {
  syntax: '<color>';
  inherits: false;
  initial-value: blue;
}

/* If --my-color is not set, it defaults to blue */
```

Key Difference: Animatable vs Non-Animatable

Aspect Regular CSS Variable Typed @property **Data Type** String (untyped) Specific type (color, angle, etc.) **Animatable** No - jumps between values Yes - smooth interpolation **Inheritance** Always inherits Configurable per property **Type Checking** None Strict - invalid values rejected **Browser Rendering** Browser doesn't optimize Browser optimizes for animation

Supported Syntax Types

Color Types

```
@property --my-color {
  syntax: '<color>';
  inherits: false;
  initial-value: red;
}
```

Numeric Types

```
@property --my-number {
  syntax: '<number>';
  inherits: false;
  initial-value: 0;
}

@property --my-integer {
  syntax: '<integer>';
  inherits: false;
  initial-value: 1;
}
```

Dimension Types

```
@property --my-length {
  syntax: '<length>';
  inherits: false;
  initial-value: 10px;
}

@property --my-angle {
  syntax: '<angle>';
  inherits: false;
  initial-value: 0deg;
}

@property --my-percentage {
  syntax: '<percentage>';
  inherits: false;
  initial-value: 50%;
}
```

URL and Image Types

```
@property --my-image {
  syntax: '<image>';
  inherits: false;
  initial-value: url('image.jpg');
}

@property --my-url {
  syntax: '<url>';
  inherits: false;
  initial-value: url('default.jpg');
}
```

Wildcard Type (Any Value)

```
@property --my-anything {
  syntax: '*';
  inherits: false;
  initial-value: auto;
}

/* Accepts any CSS value - acts like regular CSS Variable */
```

Multiple Types (with + or #)

```
/* One or more values */
@property --my-colors {
  syntax: '<color>#';
  inherits: false;
  initial-value: red, blue;
}

/* Alternatives */
@property --my-size {
  syntax: '<length> | <percentage>';
  inherits: false;
  initial-value: 100px;
}
```

🎬 Interactive @property Demos

Typed properties animate smoothly because the browser understands their data types and can interpolate values.

Hue Rotation

@property --hue: <angle>

Saturation Animation

@property --saturation: <percentage>

Lightness Animation

@property --lightness: <percentage>

Gradient Hue

Animated gradient endpoint

Practical Examples & Use Cases

1. Animatable Color Change

```
@property --button-color {
  syntax: '<color>';
  inherits: false;
  initial-value: blue;
}

@keyframes colorShift {
  0% { --button-color: blue; }
  50% { --button-color: purple; }
  100% { --button-color: blue; }
}

button {
  background: var(--button-color);
  animation: colorShift 2s infinite;
}
```

2. Animatable Length (Border Radius)

```
@property --border-size {
  syntax: '<length>';
  inherits: false;
  initial-value: 0px;
}

@keyframes radiusChange {
  0% { --border-size: 0px; }
  100% { --border-size: 50px; }
}

.shape {
  width: 100px;
  height: 100px;
  border-radius: var(--border-size);
  animation: radiusChange 2s infinite alternate;
}
```

3. Rotation with Angles

```
@property --rotation-angle {
  syntax: '<angle>';
  inherits: false;
  initial-value: 0deg;
}

@keyframes spin {
  from { --rotation-angle: 0deg; }
  to { --rotation-angle: 360deg; }
}

.spinner {
  transform: rotate(var(--rotation-angle));
  animation: spin 2s linear infinite;
}
```

4. Percentage Animations (Opacity)

```
@property --opacity-level {
  syntax: '<percentage>';
  inherits: false;
  initial-value: 100%;
}

@keyframes fadeInOut {
  0%, 100% { --opacity-level: 100%; }
  50% { --opacity-level: 0%; }
}

.fading {
  opacity: var(--opacity-level);
  animation: fadeInOut 2s infinite;
}
```

5. Smooth Number Transitions

```
@property --blur-amount {
  syntax: '<number>';
  inherits: false;
  initial-value: 0;
}

@keyframes blurEffect {
  0% { --blur-amount: 0; }
  50% { --blur-amount: 10; }
  100% { --blur-amount: 0; }
}

img {
  filter: blur(var(--blur-amount, 0)px);
  animation: blurEffect 2s infinite;
}
```

6. Theme Switching with Inheritance

```
@property --theme-color {
  syntax: '<color>';
  inherits: true; /* Inherits to children */
  initial-value: black;
}

:root {
  --theme-color: black;
}

:root.light-theme {
  --theme-color: white;
}

p {
  color: var(--theme-color);
  /* Automatically inherits from :root */
}
```

Advanced Techniques

Combining @property with calc()

```
@property --scale {
  syntax: '<number>';
  inherits: false;
  initial-value: 1;
}

.element {
  width: calc(100px * var(--scale));
  height: calc(100px * var(--scale));
  animation: scaleUp 2s infinite;
}

@keyframes scaleUp {
  0% { --scale: 1; }
  100% { --scale: 2; }
}
```

@property with Transitions

```
@property --card-shadow {
  syntax: '<length>';
  inherits: false;
  initial-value: 0px;
}

.card {
  box-shadow: 0 var(--card-shadow) 20px rgba(0,0,0,0.2);
  transition: --card-shadow 0.3s ease;
}

.card:hover {
  --card-shadow: 10px;
}
```

Multiple @property Rules

```
@property --r { syntax: '<number>'; inherits: false; initial-value: 255; }
@property --g { syntax: '<number>'; inherits: false; initial-value: 0; }
@property --b { syntax: '<number>'; inherits: false; initial-value: 0; }

.rgb-element {
  background: rgb(var(--r), var(--g), var(--b));
  animation: colorCycle 3s infinite;
}

@keyframes colorCycle {
  0% { --r: 255; --g: 0; --b: 0; }
  33% { --r: 0; --g: 255; --b: 0; }
  66% { --r: 0; --g: 0; --b: 255; }
  100% { --r: 255; --g: 0; --b: 0; }
}
```

Feature Detection

```
@supports (background: paint(something)) {
  /* @property is supported */
  @property --my-color {
    syntax: '<color>';
    inherits: false;
    initial-value: red;
  }
}

@supports not (background: paint(something)) {
  /* Fallback for browsers without @property support */
  .element { background: red; }
}
```

Browser Compatibility & Fallbacks

Chrome/Edge: Full support for @property since version 85.

Safari: Full support since version 16.4.

Firefox: Limited support. Check current status as implementation is ongoing.

IE 11/Old Browsers: No support. Provide fallback values for custom properties.

Providing Fallbacks

```
.element {
  /* Fallback for unsupported browsers */
  background: blue;
  /* @property-based styling for supported browsers */
  background: var(--animated-color, blue);
}
```

Best Practices

- **Use @property for animated values:** When you need smooth transitions or animations of custom properties.
- **Always set initial-value:** Provides a sensible default if the property isn't set.
- **Use correct syntax type:** Match the syntax to how the property will be used (color, angle, length, etc.).
- **Consider inheritance needs:** Set inherits: true only if the property should cascade to child elements.
- **Provide fallbacks:** Use @supports or fallback values for older browsers.
- **Test in target browsers:** @property support is still emerging - test thoroughly.
- **Use meaningful property names:** Start with double hyphen and use descriptive names.
- **Document your @property rules:** Add comments explaining what each property is for.

Common Mistakes to Avoid

- **Missing initial-value:** The initial-value descriptor is required. Without it, the @property rule fails.
- **Wrong syntax type:** Using syntax: '<color>' with a length value will cause issues.
- **Not using var() in CSS:** Defining @property is just registration - you still need var() in your CSS.
- **Invalid values in animations:** Keyframe values must match the declared syntax type.
- **No @supports fallback:** Assuming @property is supported everywhere without testing.
- **Mixing syntax notations:** Be consistent with quotes around syntax values.
- **Using on :root without inheritance flag:** Doesn't automatically inherit unless inherits: true.
- **Complex animations without testing:** Multiple @property rules animating together need careful testing.

Ideal Use Cases for @property

- **Animated gradients:** Smoothly animate gradient endpoints using angles and colors.
- **Color transitions:** Animate color changes smoothly instead of jumping.
- **Dynamic sizing:** Animate sizes, radii, and dimensions smoothly.
- **Complex animations:** Multi-part animations using multiple @property rules.
- **Interactive effects:** Smooth transitions on user interaction (hover, click).
- **Theming systems:** Typed properties work well with theme switching.
- **Data visualization:** Animate numeric values smoothly for charts and graphs.
- **Advanced UI effects:** Create sophisticated animations that CSS Variables alone can't achieve.

Quick Reference

Basic @property Template

```
@property --my-property {
  syntax: '<type>';
  inherits: false;
  initial-value: <default>;
}
```

Common Patterns

```
/* Color animation */
@property --color { syntax: '<color>'; inherits: false; initial-value: red; }

/* Angle rotation */
@property --angle { syntax: '<angle>'; inherits: false; initial-value: 0deg; }

/* Length sizing */
@property --size { syntax: '<length>'; inherits: false; initial-value: 0px; }

/* Number/percentage */
@property --opacity { syntax: '<percentage>'; inherits: false; initial-value: 100%; }
```

Animation Usage

```
@keyframes myAnimation {
  0% { --my-property: value1; }
  100% { --my-property: value2; }
}

.element {
  /* Use the property */
  property-name: var(--my-property);
  /* Animate it */
  animation: myAnimation 2s infinite;
}
```